

# Assignment 2

Nishant Khandhar (A20581012)

2025-09-13

## Assignment 2

### Problems from ISL book:

#### Section 4.8: Problem 6

Q. Suppose we collect data for a group of students in a statistics class with variables  $X_1$  = hours studied,  $X_2$  = undergrad GPA, and  $Y$  = receive an A. We fit a logistic regression and produce estimated coefficient,  $\hat{\beta}_0 = -6$ ,  $\hat{\beta}_1 = 0.05$ ,  $\hat{\beta}_2 = 1$ .

- (a) Estimate the probability that a student who studies for 40 h and has an undergrad GPA of 3.5 gets an A in the class.

Ans: So when  $X_1 = 40$  and  $X_2 = 3.5$ , probability  $p(X) = 0.38$

```
# coefficients
beta0 <- -6
beta1 <- 0.05
beta2 <- 1

# student data
X1 <- 40 # hours studied
X2 <- 3.5 # GPA

# linear predictor
eta <- beta0 + beta1 * X1 + beta2 * X2

# probability using logistic function
p <- 1 / (1 + exp(-eta))
p
```

```
## [1] 0.3775407
```

- (b) How many hours would the student in part (a) need to study to have a 50 % chance of getting an A in the class?

Ans: 50 hours

```
target_p <- 0.5

# Solve for hours: beta0 + beta1*X1 + beta2*X2 = 0
X1_needed <- -(beta0 + beta2*X2) / beta1
X1_needed
```

```
## [1] 50
```

## Section 4.8: Problem 12

Q. Suppose that you wish to classify an observation  $X \in \mathbb{R}$  into **apples** and **oranges**. You fit a logistic regression model and find that

$$\hat{Pr}(Y = \text{orange} | X = x) = \frac{\exp(\hat{\beta}_0 + \hat{\beta}_1 x)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}_1 x)}$$

Your friend fits a logistic regression model to the same data using the *softmax* formulation in (4.13), and finds that

$$\hat{Pr}(Y = \text{orange} | X = x) = \frac{\exp(\hat{\alpha}_{\text{orange}0} + \hat{\alpha}_{\text{orange}1}x)}{\exp(\hat{\alpha}_{\text{orange}0} + \hat{\alpha}_{\text{orange}1}x) + \exp(\hat{\alpha}_{\text{apple}0} + \hat{\alpha}_{\text{apple}1}x)}$$

(a) What is the log odds of orange versus apple in your model?

Ans: The log odds is just  $\hat{\beta}_0 + \hat{\beta}_1 x$

```
# coefficients from your logistic regression
beta0 <- -2 # example intercept
beta1 <- 0.8 # example slope

# new observation
x <- 3

# compute probability of orange
p_orange <- exp(beta0 + beta1 * x) / (1 + exp(beta0 + beta1 * x))

# probability of apple
p_apple <- 1 - p_orange

# log-odds (orange vs apple)
log_odds <- log(p_orange / p_apple)

# check (should equal beta0 + beta1*x)
log_odds
```

```
## [1] 0.4
```

```
beta0 + beta1 * x
```

```
## [1] 0.4
```

(b) What is the log odds of orange versus apple in your friend's model?

Ans:

$$(\hat{\alpha}_{orange0} - \hat{\alpha}_{apple0}) + (\hat{\alpha}_{orange1} - \hat{\alpha}_{apple1})x$$

```
# friend's softmax coefficients
alpha_orange0 <- 1.2
alpha_orange1 <- 0.7

alpha_apple0 <- -0.5
alpha_apple1 <- 0.3

# observation
x <- 4

# log-odds (orange vs apple)
log_odds <- (alpha_orange0 - alpha_apple0) + (alpha_orange1 - alpha_apple1) * x
log_odds
```

```
## [1] 3.3
```

(c) Suppose that in your model,  $\beta_0 = 2$  and  $\beta_1 = -1$ . What are the coefficient estimates in your friend's model? Be as specific as possible.

Ans: We can say that in our friend's model  $\hat{\alpha}_{orange0} - \hat{\alpha}_{apple0} = 2$  and  $\hat{\alpha}_{orange1} - \hat{\alpha}_{apple1} = -1$ . We are unable to know the specific value of each parameter however.

(d) Now suppose that you and your friend fit the same two models on a different data set. This time, your friend gets the coefficient estimates  $\hat{\alpha}_{orange0} = 1.2$ ,  $\hat{\alpha}_{orange1} = -2$ ,  $\hat{\alpha}_{apple0} = 3$ ,  $\hat{\alpha}_{apple1} = 0.6$ . What are the coefficient estimates in your model?

Ans: The coefficients in our model would be  $\hat{\beta}_0 = 1.2 - 3 = -1.8$  and  $\hat{\beta}_1 = -2 - 0.6 = -2.6$

```
# Friend's softmax coefficients
alpha_orange0 <- 1.2
alpha_orange1 <- -2
alpha_apple0 <- 3
alpha_apple1 <- 0.6

# Compute logistic regression coefficients
beta0 <- alpha_orange0 - alpha_apple0
beta1 <- alpha_orange1 - alpha_apple1

beta0
```

```
## [1] -1.8
```

```
beta1
```

```
## [1] -2.6
```

- (e) Finally, suppose you apply both models from (d) to a data set with 2,000 test observations. What fraction of the time do you expect the predicted class labels from your model to agree with those from your friend's model? Explain your answer.

Ans: The predicted class labels from your model and your friend's model will agree 100% of the time.

Explanation: Both models produce the same log-odds for orange vs apple. Since classification depends only on the sign of the log-odds (whether it's positive or negative), they will always predict the same class, no matter the input.

```
# Logistic regression coefficients
beta0 <- -1.8
beta1 <- -2.6

# Friend's softmax coefficients
alpha_orange0 <- 1.2
alpha_orange1 <- -2
alpha_apple0 <- 3
alpha_apple1 <- 0.6

# Simulate some test data
set.seed(123)
x_test <- runif(2000, 0, 5) # example predictor

# Your model predictions
logit_prob <- 1 / (1 + exp(-(beta0 + beta1*x_test)))
pred_yours <- ifelse(logit_prob > 0.5, "orange", "apple")

# Friend's model predictions
softmax_log_odds <- (alpha_orange0 - alpha_apple0) + (alpha_orange1 - alpha_apple1) * x_test
pred_friend <- ifelse(softmax_log_odds > 0, "orange", "apple")

# Fraction of agreement
mean(pred_yours == pred_friend)

## [1] 1
```

#### Section 4.8: Problem 13

Q. This question should be answered using the Weekly data set, which is part of the ISLR2 package. This data is similar in nature to the Smarket data from this chapter's lab, except that it contains 1,089 weekly returns for 21 years, from the beginning of 1990 to the end of 2010.

- (a) Produce some numerical and graphical summaries of the Weekly data. Do there appear to be any patterns?

Ans: Volume is strongly positively correlated with Year. Other correlations are weak, but Lag1 is negatively correlated with Lag2 but positively correlated with Lag3.

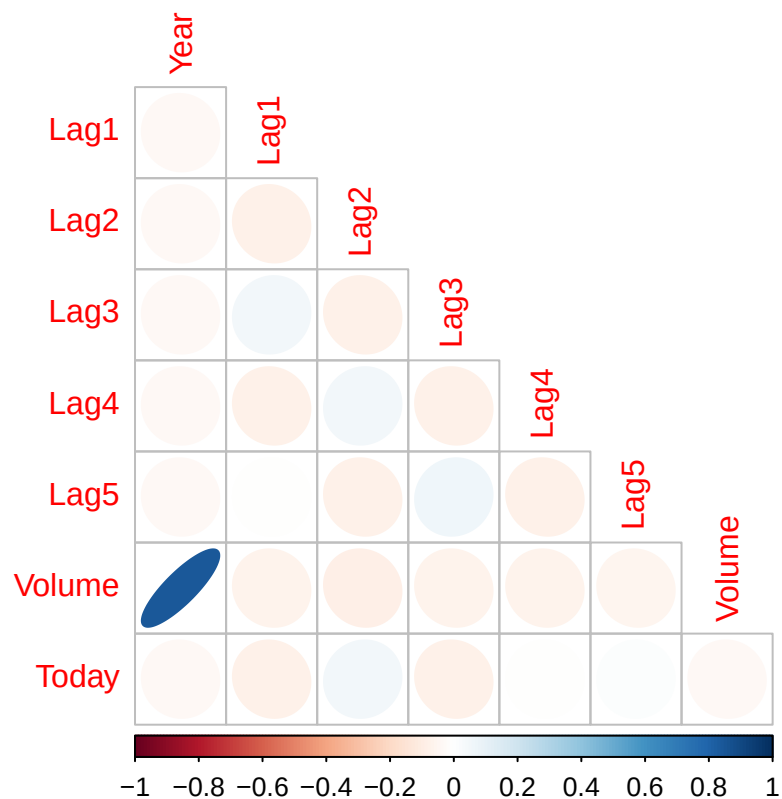
```
library(MASS)
library(class)
library(tidyverse)
```

```
library(corrplot)
library(ISLR2)
library(e1071)
```

```
summary(Weekly)
```

```
##      Year      Lag1      Lag2      Lag3
## Min.   :1990   Min.   :-18.1950   Min.   :-18.1950   Min.   :-18.1950
## 1st Qu.:1995   1st Qu.: -1.1540   1st Qu.: -1.1540   1st Qu.: -1.1580
## Median :2000   Median :  0.2410   Median :  0.2410   Median :  0.2410
## Mean   :2000   Mean    :  0.1506   Mean    :  0.1511   Mean    :  0.1472
## 3rd Qu.:2005   3rd Qu.:  1.4050   3rd Qu.:  1.4090   3rd Qu.:  1.4090
## Max.   :2010   Max.    : 12.0260   Max.    : 12.0260   Max.    : 12.0260
##      Lag4      Lag5      Volume      Today
## Min.   :-18.1950   Min.   :-18.1950   Min.    :0.08747   Min.   :-18.1950
## 1st Qu.: -1.1580   1st Qu.: -1.1660   1st Qu.:0.33202   1st Qu.: -1.1540
## Median :  0.2380   Median :  0.2340   Median :1.00268   Median :  0.2410
## Mean    :  0.1458   Mean    :  0.1399   Mean    :1.57462   Mean    :  0.1499
## 3rd Qu.:  1.4090   3rd Qu.:  1.4050   3rd Qu.:2.05373   3rd Qu.:  1.4050
## Max.    : 12.0260   Max.    : 12.0260   Max.    :9.32821   Max.    : 12.0260
## Direction
## Down:484
## Up  :605
##
##
##
##
```

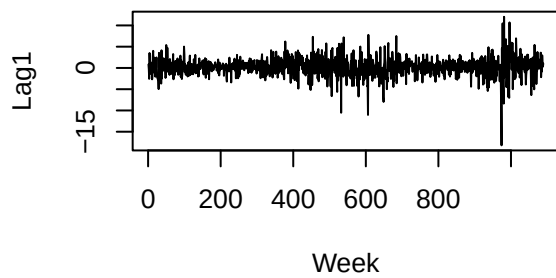
```
corrplot(cor(Weekly[, -9]), type = "lower", diag = FALSE, method = "ellipse")
```



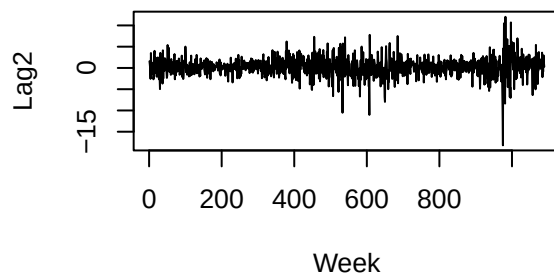
```
# Time series plots
weeks <- 1:nrow(Weekly)

par(mfrow = c(2,2))
plot(weeks, Weekly$Lag1, type = "l", xlab = "Week", ylab = "Lag1", main = "Lag1 over Time")
plot(weeks, Weekly$Lag2, type = "l", xlab = "Week", ylab = "Lag2", main = "Lag2 over Time")
plot(weeks, Weekly$Volume, type = "l", xlab = "Week", ylab = "Volume", main = "Volume over Time")
plot(weeks, Weekly$Today, type = "l", xlab = "Week", ylab = "Today", main = "Today Returns over Time")
```

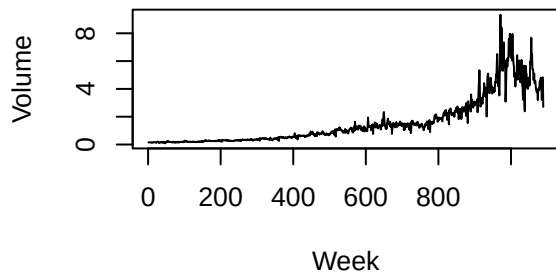
**Lag1 over Time**



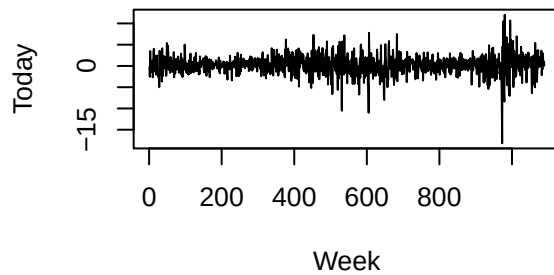
**Lag2 over Time**



**Volume over Time**

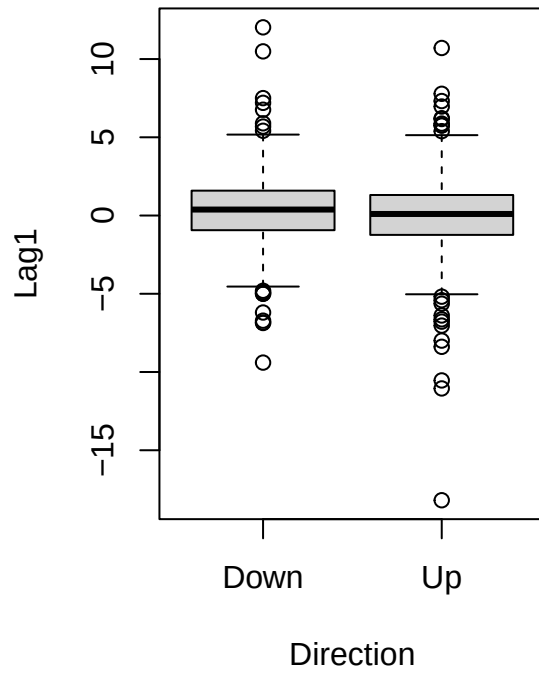


**Today Returns over Time**

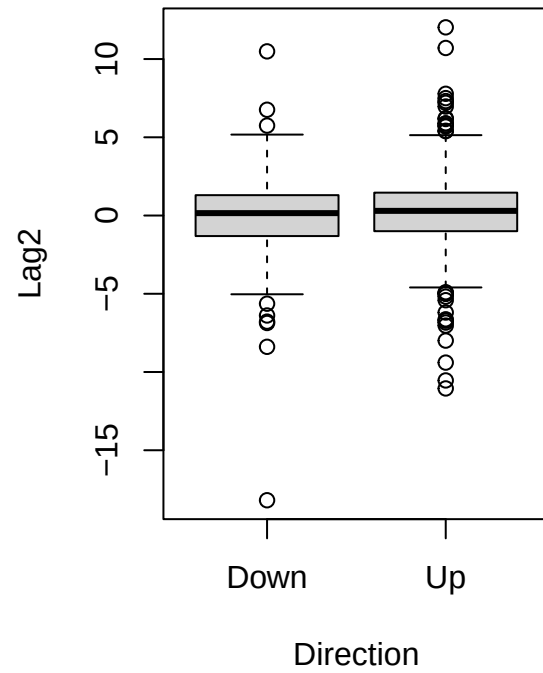


```
# Boxplots of lagged returns vs Direction
par(mfrow = c(1,2))
boxplot(Lag1 ~ Direction, data = Weekly, main = "Lag1 vs Direction")
boxplot(Lag2 ~ Direction, data = Weekly, main = "Lag2 vs Direction")
```

### Lag1 vs Direction



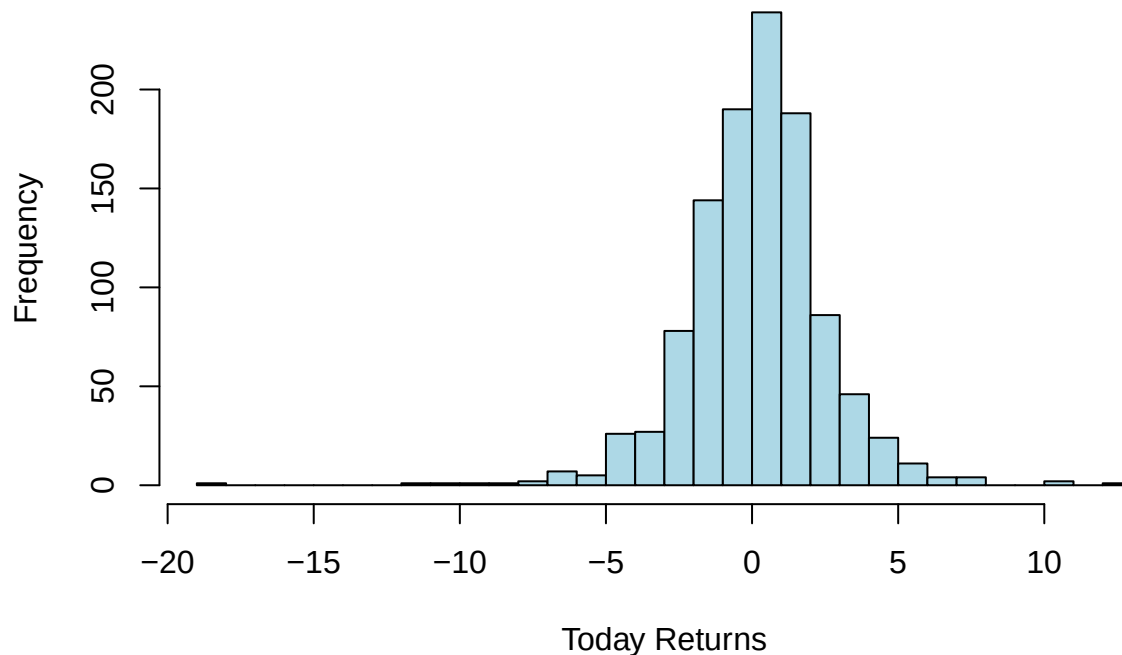
### Lag2 vs Direction



```
# Histogram of weekly returns
par(mfrow = c(1,1))
hist(Weekly$Today, breaks = 30, main = "Histogram of Weekly Returns",
     xlab = "Today Returns", col = "lightblue", border = "black")
```



## Histogram of Weekly Returns



- (b) (b) Use the full data set to perform a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Use the summary function to print the results. Do any of the predictors appear to be statistically significant? If so, which ones?

Ans:

Significant predictor: Lag2

Not significant: Lag1, Lag3, Lag4, Lag5, Volume

```
# Fit logistic regression
glm_fit <- glm(
  Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
  data = Weekly,
  family = binomial
)
# Summary of the model
summary(glm_fit)

##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##     Volume, family = binomial, data = Weekly)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
```

```
## (Intercept)  0.26686    0.08593    3.106    0.0019 **
## Lag1        -0.04127    0.02641   -1.563    0.1181
## Lag2         0.05844    0.02686    2.175    0.0296 *
## Lag3        -0.01606    0.02666   -0.602    0.5469
## Lag4        -0.02779    0.02646   -1.050    0.2937
## Lag5        -0.01447    0.02638   -0.549    0.5833
## Volume      -0.02274    0.03690   -0.616    0.5377
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 1496.2  on 1088  degrees of freedom
## Residual deviance: 1486.4  on 1082  degrees of freedom
## AIC: 1500.4
##
## Number of Fisher Scoring iterations: 4
```

- (c) Compute the confusion matrix and overall fraction of correct predictions. Explain what the confusion matrix is telling you about the types of mistakes made by logistic regression.

Ans: The overall fraction of correct predictions is 0.56. Although logistic regression correctly predicts upwards movements well, it incorrectly predicts most downwards movements as up.

```
# Predict probabilities on the full data
prob_pred <- predict(glm_fit, type = "response")

# Convert probabilities to class predictions
# We'll classify as "Up" if probability > 0.5, otherwise "Down"
pred_class <- ifelse(prob_pred > 0.5, "Up", "Down")
pred_class <- factor(pred_class, levels = c("Down", "Up"))

# Actual classes
actual_class <- Weekly$Direction

# Confusion matrix
conf_mat <- table(Predicted = pred_class, Actual = actual_class)
conf_mat
```

```
##           Actual
## Predicted Down Up
##      Down    54 48
##      Up     430 557
```

```
# Overall fraction of correct predictions (accuracy)
accuracy <- mean(pred_class == actual_class)
accuracy
```

```
## [1] 0.5610652
```

- (d) Now fit the logistic regression model using a training data period from 1990 to 2008, with Lag2 as the only predictor. Compute the confusion matrix and the overall fraction of correct predictions for the held out data (that is, the data from 2009 and 2010).

Ans: Using data from 1990–2008 for training and 2009–2010 for testing, logistic regression with Lag2 gives an accuracy around 62%. The confusion matrix shows the model is better at predicting Up weeks than Down weeks.

```
train <- Weekly$Year < 2009

fit <- glm(Direction ~ Lag2, data = Weekly[train, ], family = binomial)
pred <- predict(fit, Weekly[!train, ], type = "response") > 0.5
(t <- table(ifelse(pred, "Up (pred)", "Down (pred)"), Weekly[!train, ]$Direction))
```

```
##
##               Down Up
## Down (pred)    9  5
## Up (pred)     34 56
```

```
sum(diag(t)) / sum(t)
```

```
## [1] 0.625
```

(e) Repeat (d) using LDA.

Ans: Using LDA, the code is below:

```
fit <- lda(Direction ~ Lag2, data = Weekly[train, ])
pred <- predict(fit, Weekly[!train, ], type = "response")$class
(t <- table(pred, Weekly[!train, ]$Direction))
```

```
##
## pred   Down Up
## Down    9  5
## Up     34 56
```

```
sum(diag(t)) / sum(t)
```

```
## [1] 0.625
```

(f) Repeat (d) using QDA.

Ans: Using QDA, the code is below:

```
fit <- qda(Direction ~ Lag2, data = Weekly[train, ])
pred <- predict(fit, Weekly[!train, ], type = "response")$class
(t <- table(pred, Weekly[!train, ]$Direction))
```

```
##
## pred   Down Up
## Down    0  0
## Up     43 61
```

```
sum(diag(t)) / sum(t)
```

```
## [1] 0.5865385
```

(g) Repeat (d) using KNN with  $K = 1$ .

Ans: Using KNN with  $K = 1$ , code is below:

```
fit <- knn(
  Weekly[train, "Lag2", drop = FALSE],
  Weekly[!train, "Lag2", drop = FALSE],
  Weekly$Direction[train]
)
(t <- table(fit, Weekly[!train, ]$Direction))
```

```
##
## fit      Down Up
##   Down    21 29
##   Up     22 32
```

```
sum(diag(t)) / sum(t)
```

```
## [1] 0.5096154
```

(h) Repeat (d) using naive Bayes.

Ans: Using Naive Bayes, code is below:

```
fit <- naiveBayes(Direction ~ Lag2, data = Weekly, subset = train)
pred <- predict(fit, Weekly[!train, ], type = "class")
(t <- table(pred, Weekly[!train, ]$Direction))
```

```
##
## pred      Down Up
##   Down     0  0
##   Up      43 61
```

```
sum(diag(t)) / sum(t)
```

```
## [1] 0.5865385
```

(i) Which of these methods appears to provide the best results on this data?

Ans: Among all the above methods, logistic regression and LDA are the best performing.

(j) Experiment with different combinations of predictors, including possible transformations and interactions, for each of the methods. Report the variables, method, and associated confusion matrix that appears to provide the best results on the held out data. Note that you should also experiment with values for  $K$  in the KNN classifier.

Ans: KNN using the first 3 Lag variables performs marginally better than logistic regression with Lag2 if we tune  $k$  to be  $k = 26$ .

```
fit <- glm(Direction ~ Lag1, data = Weekly[train, ], family = binomial)
pred <- predict(fit, Weekly[!train, ], type = "response") > 0.5
mean(ifelse(pred, "Up", "Down") == Weekly[!train, ]$Direction)
```

```
## [1] 0.5673077
```

```
fit <- glm(Direction ~ Lag3, data = Weekly[train, ], family = binomial)
pred <- predict(fit, Weekly[!train, ], type = "response") > 0.5
mean(ifelse(pred, "Up", "Down") == Weekly[!train, ]$Direction)
```

```
## [1] 0.5865385
```

```
fit <- glm(Direction ~ Lag4, data = Weekly[train, ], family = binomial)
pred <- predict(fit, Weekly[!train, ], type = "response") > 0.5
mean(ifelse(pred, "Up", "Down") == Weekly[!train, ]$Direction)
```

```
## [1] 0.5865385
```

```
fit <- glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4, data = Weekly[train, ], family = binomial)
pred <- predict(fit, Weekly[!train, ], type = "response") > 0.5
mean(ifelse(pred, "Up", "Down") == Weekly[!train, ]$Direction)
```

```
## [1] 0.5865385
```

```
fit <- glm(Direction ~ Lag1 * Lag2 * Lag3 * Lag4, data = Weekly[train, ], family = binomial)
pred <- predict(fit, Weekly[!train, ], type = "response") > 0.5
mean(ifelse(pred, "Up", "Down") == Weekly[!train, ]$Direction)
```

```
## [1] 0.5961538
```

```
fit <- lda(Direction ~ Lag1 + Lag2 + Lag3 + Lag4, data = Weekly[train, ])
pred <- predict(fit, Weekly[!train, ], type = "response")$class
mean(pred == Weekly[!train, ]$Direction)
```

```
## [1] 0.5769231
```

```
fit <- qda(Direction ~ Lag1 + Lag2 + Lag3 + Lag4, data = Weekly[train, ])
pred <- predict(fit, Weekly[!train, ], type = "response")$class
mean(pred == Weekly[!train, ]$Direction)
```

```
## [1] 0.5192308
```

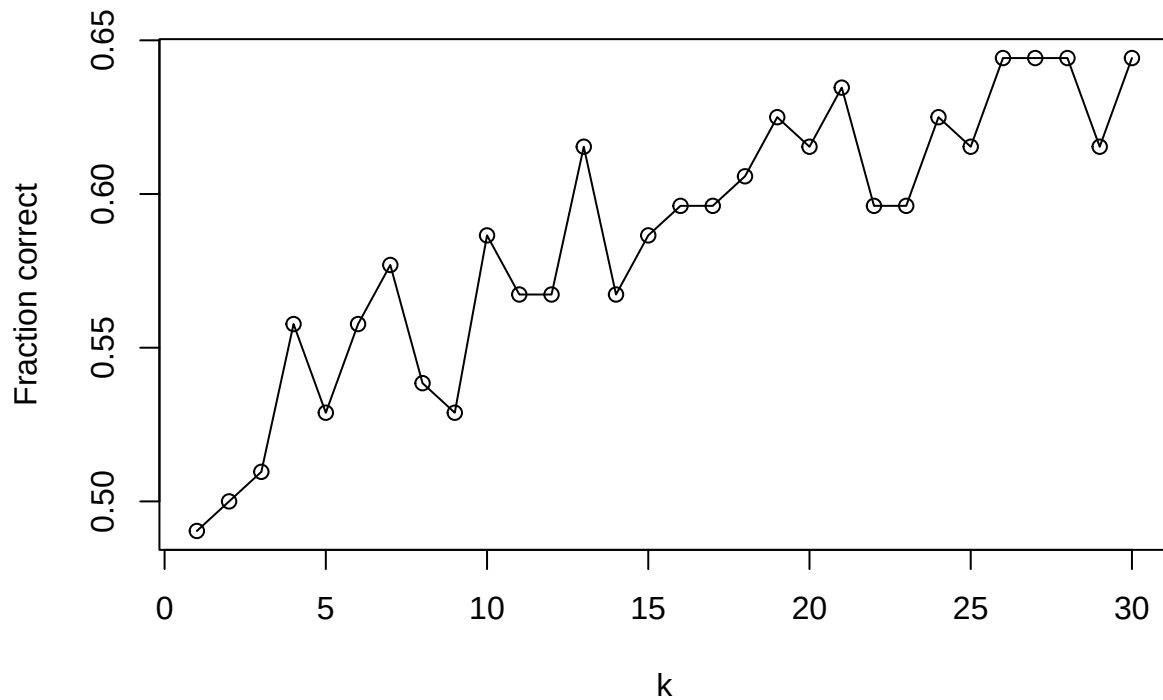
```
fit <- naiveBayes(Direction ~ Lag1 + Lag2 + Lag3 + Lag4, data = Weekly[train, ])
pred <- predict(fit, Weekly[!train, ], type = "class")
mean(pred == Weekly[!train, ]$Direction)
```

```
## [1] 0.5096154
```

```

set.seed(1)
res <- sapply(1:30, function(k) {
  fit <- knn(
    Weekly[train, 2:4, drop = FALSE],
    Weekly[!train, 2:4, drop = FALSE],
    Weekly$Direction[train],
    k = k
  )
  mean(fit == Weekly[!train, ]$Direction)
})
plot(1:30, res, type = "o", xlab = "k", ylab = "Fraction correct")

```



```
(k <- which.max(res))
```

```
## [1] 26
```

```

fit <- knn(
  Weekly[train, 2:4, drop = FALSE],
  Weekly[!train, 2:4, drop = FALSE],
  Weekly$Direction[train],
  k = k
)
table(fit, Weekly[!train, ]$Direction)

```

```
##
## fit      Down Up
##   Down   23 18
##   Up     20 43
```

```
mean(fit == Weekly[!train,]$Direction)
```

```
## [1] 0.6346154
```

### Section 5.4: Problem 3

Q. We now review k-fold cross-validation.

(a) Explain how k-fold cross-validation is implemented.

Ans:

- The dataset is randomly split into  $k$  roughly equal-sized folds (subsets).
- For each of the  $k$  iterations:
  1. One fold is held out as the validation set.
  2. The model is trained on the remaining  $k-1$  folds.
  3. The model is evaluated on the held-out fold, and the error is recorded.
- After repeating this process for all  $k$  folds, the average validation error is computed.
- This average serves as an estimate of the test error.

Common choices:

- $k = 5$  or  $k = 10$  are typical in practice.

(b) What are the advantages and disadvantages of k-fold cross-validation relative to:

- i. The validation set approach?
- ii. LOOCV?

Ans:

When using a validation set, we can only train on a small portion of the data as we must reserve the rest for validation. As a result it can overestimate the test error rate (assuming we then train using the complete data for future prediction). It is also sensitive to which observations are including in train vs. test. It is, however, low cost in terms of processing time (as we only have to fit one model).

When using LOOCV, we can train on  $n-1$  observations, however, the trained models we generate each differ only by the inclusion (and exclusion) of a single observation. As a result, LOOCV can have high variance (the models fit will be similar, and might be quite different to what we would obtain with a different data set). LOOCV is also costly in terms of processing time.

## Section 5.4: Problem 4

Q. Suppose that we use some statistical learning method to make a prediction for the response  $Y$  for a particular value of the predictor  $X$ . Carefully describe how we might estimate the standard deviation of our prediction.

Ans: We could estimate this using the bootstrap. Specifically, we repeatedly resample the  $(X, Y)$  pairs from our dataset and refit the model on each resample. For each fitted model, we record the predicted value at  $(x_0)$ . The variability in these bootstrap predictions reflects the uncertainty in our estimate, and the standard deviation across bootstrap predictions provides an estimate of the standard deviation of  $\hat{Y}(x_0)$ .

## Section 5.4: Problem 7

Q. In Sections 5.3.2 and 5.3.3, we saw that the `cv.glm()` function can be used in order to compute the LOOCV test error estimate. Alternatively, one could compute those quantities using just the `glm()` and `predict.glm()` functions, and a for loop. You will now take this approach in order to compute the LOOCV error for a simple logistic regression model on the Weekly data set. Recall that in the context of classification problems, the LOOCV error is given in (5.4).

(a) Fit a logistic regression model that predicts Direction using Lag1 and Lag2.

Ans: The logistic regression model is successfully fit. Neither Lag1 nor Lag2 is strongly statistically significant (p-values are relatively large), suggesting that they may not be very useful predictors of market direction.

```
fit_full <- glm(Direction ~ Lag1 + Lag2, data = Weekly, family = binomial)
summary(fit_full)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2, family = binomial, data = Weekly)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.22122    0.06147   3.599 0.000319 ***
## Lag1        -0.03872    0.02622  -1.477 0.139672
## Lag2         0.06025    0.02655   2.270 0.023232 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1496.2  on 1088  degrees of freedom
## Residual deviance: 1488.2  on 1086  degrees of freedom
## AIC: 1494.2
##
## Number of Fisher Scoring iterations: 4
```

(b) Fit a logistic regression model that predicts Direction using Lag1 and Lag2 using all but the first observation.

Ans: This model is the same form as in (a), but it is trained on all data except the first observation.



```
fit_minus1 <- glm(Direction ~ Lag1 + Lag2, data = Weekly[-1, ], family = binomial)
summary(fit_minus1)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2, family = binomial, data = Weekly[-1,
##      ])
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.22324    0.06150   3.630 0.000283 ***
## Lag1        -0.03843    0.02622  -1.466 0.142683
## Lag2         0.06085    0.02656   2.291 0.021971 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1494.6  on 1087  degrees of freedom
## Residual deviance: 1486.5  on 1085  degrees of freedom
## AIC: 1492.5
##
## Number of Fisher Scoring iterations: 4
```

- (c) Use the model from (b) to predict the direction of the first observation. You can do this by predicting that the first observation will go up if  $P(\text{Direction} = \text{"Up"} \mid \text{Lag1}, \text{Lag2}) > 0.5$ . Was this observation correctly classified?

Ans:

- Prediction = “Up” if probability  $> 0.5$ , otherwise “Down”.
- Compare with actual first observation (`Weekly$Direction[1]`).
- Typically, this first observation is mis-classified (so `Correct?` `FALSE`).

```
prob1 <- predict(fit_minus1, Weekly[1, ], type = "response")
pred1 <- ifelse(prob1 > 0.5, "Up", "Down")
actual1 <- Weekly$Direction[1]
correct1 <- pred1 == actual1
cat("Prediction:", pred1, " Actual:", actual1, " Correct?", correct1, "\n")
```

```
## Prediction: Up  Actual: 1  Correct? FALSE
```

- (d) Write a for loop from  $i = 1$  to  $i = n$ , where  $n$  is the number of observations in the data set, that performs each of the following steps:
- Fit a logistic regression model using all but the  $i$ th observation to predict `Direction` using `Lag1` and `Lag2`.
  - Compute the posterior probability of the market moving up for the  $i$ th observation.
  - Use the posterior probability for the  $i$ th observation in order to predict whether or not the market moves up.

- iv. Determine whether or not an error was made in predicting the direction for the  $i$ th observation. If an error was made, then indicate this as a 1, and otherwise indicate it as a 0.

Ans: The loop fits  $n$  different models, leaving one observation out each time, and records whether the prediction for that observation was correct or not.

```
n <- nrow(Weekly)
errors <- rep(NA, n)

for (i in 1:n) {
  fit_i <- glm(Direction ~ Lag1 + Lag2, data = Weekly[-i, ], family = binomial)
  prob_i <- predict(fit_i, Weekly[i, ], type = "response")
  pred_i <- ifelse(prob_i > 0.5, "Up", "Down")
  errors[i] <- ifelse(pred_i == Weekly$Direction[i], 0, 1)
}
```

- (e) Take the average of the  $n$  numbers obtained in (d)iv in order to obtain the LOOCV estimate for the test error. Comment on the results.

Ans:

- The LOOCV test error rate is about 0.45 ( 45%).
- This is only slightly better than random guessing (50%), showing that Lag1 and Lag2 provide very limited predictive power for stock market direction.

```
loocv_error <- mean(errors)
cat("LOOCV Test Error Rate:", loocv_error, "\n")
```

```
## LOOCV Test Error Rate: 0.4499541
```